

The Biogeochemical Model Database

Python packages for **model intercomparison**

Markus Müller

September 16, 2025

Overview

bgc_md2 Screenshots

- Views

- Single Model Inspection

User Interface and Queries

- Invisible Graphs

- Making them Visible

Structure

- Classes *and* Functions

- A Record

- Relation to other Python Packages

Non trivial diagnostic variables

- Transient age and transit time distributions for nonlinear nonautonomous compartmental models

Application to model intercomparison

- Goals

- Numeric comparison

Jupyterwidget for a subset of models bgc_md2

A screenshot of a JupyterLab interface showing a Jupyter widget for a subset of models. The browser address bar indicates the local host path: `localhost:8888/notebooks/prototypes/widgets/TECOModelComparison.ipynb`. The JupyterLab title bar shows the file name: `TECOModelComparison` and the last checkpoint: `Last Checkpoint: 11/16/2021 (unsaved changes)`. The JupyterLab toolbar shows the `Run` button and the `Volta` button. The widget interface displays a list of models with expand/collapse buttons:

- `cable_general`
- `TECOMm`
- `TECO`
- `Lux2012TE`
- `collapse`

The `collapse` section is expanded, showing a network diagram and a table of parameters. The network diagram illustrates the relationships between various carbon pools and fluxes, with nodes labeled `C_wood`, `C_foliage`, `C_passsom`, `C_slowsom`, `C_roots`, `C_metlit`, and `C_stlit`. The table below the diagram lists parameters and their values:

Parameter	Value
$27G_{B_{top}}(1-e^{-t/2})(C_{top}-C_{top}^0)$	0.254
$27G_{B_{top}}(1-e^{-t/2})(C_{top}-C_{top}^0)$	0.254
$27G_{B_{top}}(1-e^{-t/2})(C_{top}-C_{top}^0)$	0.254
$27G_{B_{top}}(1-e^{-t/2})(C_{top}-C_{top}^0)$	0.254
0	0
0	0

Analysis with symbolic tools ...

inspectModel Last Checkpoint: a minute ago (autosaved)

File Edit View Insert Cell Kernel Widgets Help

Run Code

Out [3]:

linear no state dependence undetermined
nonlinear

```
In [28]: mvs.get_CompartmentalMatrix()
Out[28]:
```

$-f_{\text{loss}2\text{DPM}} - f_{\text{loss}2\text{RPM}}$	0	0	0	0	0	0
0	$-f_{\text{wood}2\text{DPM}} - f_{\text{wood}2\text{RPM}}$	0	0	0	0	0
0	0	$-f_{\text{root}2\text{DPM}} - f_{\text{root}2\text{RPM}}$	0	0	0	0
$f_{\text{loss}2\text{DPM}}$	$f_{\text{wood}2\text{DPM}}$	$f_{\text{root}2\text{DPM}}$	$-(f_{\text{DPM}2\text{BIO}} + f_{\text{DPM}2\text{BUM}} + f_{\text{DPM}2\text{Q}})\tilde{Q}$	0	0	0
$f_{\text{loss}2\text{RPM}}$	$f_{\text{wood}2\text{RPM}}$	$f_{\text{root}2\text{RPM}}$	0	$-(f_{\text{DPM}2\text{BIO}} + f_{\text{DPM}2\text{BUM}} + f_{\text{DPM}2\text{Q}})\tilde{Q}$	0	0
0	0	0	$f_{\text{DPM}2\text{BIO}}\tilde{Q}$	$f_{\text{DPM}2\text{BIO}}\tilde{Q}$	$-(f_{\text{BIO}2\text{BUM}} + f_{\text{BIO}2\text{Q}})\tilde{Q}$	$f_{\text{BIO}2\text{BIO}}\tilde{Q}$
0	0	0	$f_{\text{DPM}2\text{BUM}}\tilde{Q}$	$f_{\text{DPM}2\text{BUM}}\tilde{Q}$	$f_{\text{BIO}2\text{BUM}}\tilde{Q}$	$-(f_{\text{DPM}2\text{BIO}} + f_{\text{DPM}2\text{Q}})\tilde{Q}$

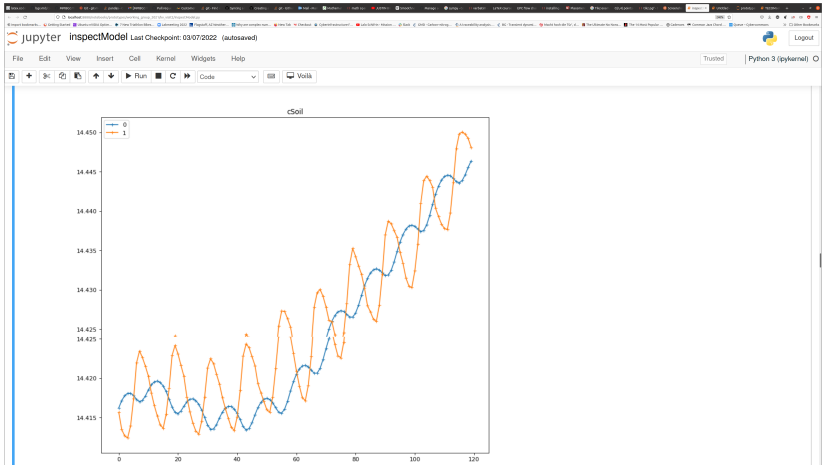
```
In [ ]: mvs.get_BibInfo
## get_CompartmentalMatrix
## get_InfluxesBySymbol
Main get_InputTuple
get_InternalFluxesBySymbol
get_OutFluxesBySymbol
get_SmoothReservoirModel
In [4]: imp
# Rq get_StateVariableTuple
with not_TimeControl
```

file in your model folder:

name: "trendy-v9", password: "qcb-2020", dataPath: "/path/to/data/folder")

sth from config.json file
len(r) as f:

... or numerically



Userinterface using computability graphs

The screenshot shows a Jupyter Notebook interface with the title "inspectModel". The interface includes a file explorer, a code editor, and an output area. A dropdown menu is open, displaying a list of suggested methods: `computable_mvar_types`, `computers`, `get_BibInfo`, `get_CompartmentalMatrix`, `get_InFluxesBySymbol`, `get_InputTuple`, `get_InternalFluxesBySymbol`, `get_OutFluxesBySymbol`, `get_SmoothReservoirModel`, and `get_StateVariableTuple`. The code editor shows the following input:

```
In [ ]: mvs.
```

The output area displays the following text:

```
In [3]: # we can also plot a picture
h.compartmental_graph(mvs)
```

Out[3]:

A legend indicates the types of nodes in the graph:

- linear (blue square)
- nonlinear (green square)
- no state dependence (red square)
- undetermined (grey square)

The graph itself shows two nodes: `Croot` (blue circle) and `Cleafitter` (blue circle). Arrows indicate the flow of information between them: a red arrow from `Croot` to `Cleafitter`, and a blue arrow from `Cleafitter` to `Croot`.

Figure: Suggested methods automatically created by a graph library

Finding what's missing

given a set of

functions:

$a(i)$, $b(c,d)$, $b(e,f)$,

$c(b)$, $d(b)$, $d(g,h)$,

$e(b)$, $f(b)$ and the

target variable **B**

e.g.

CompartmentalMatrix,

The algorithm

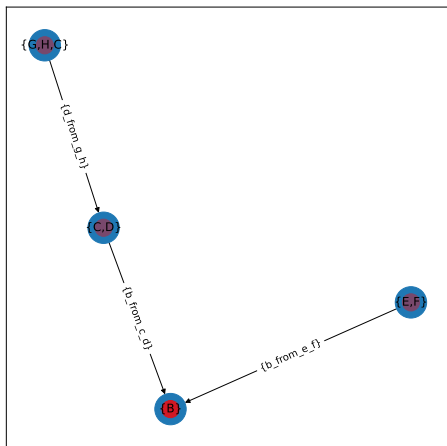
computes all

possible

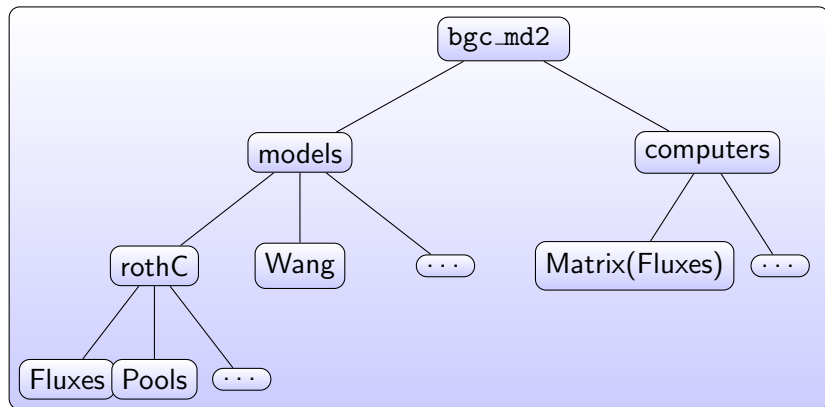
combinations and

paths from which **B**

can be computed.



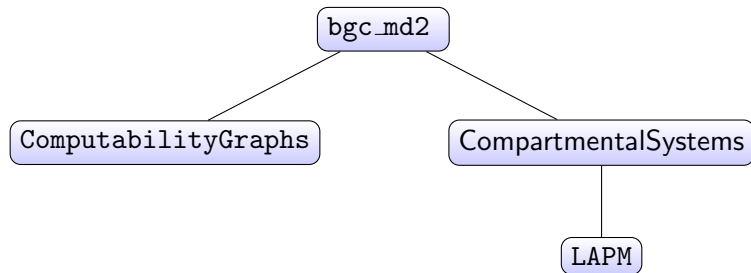
Internal Structure of bgc_md2



Database records are python modules

```
1 from sympy import Symbol, Function
2 from ComputabilityGraphs.OHVS import OHVS
3 from bgc_md2.helper import models, computers
4 from bgc_md2.models.BibInfo import BibInfo
5 from bgc_md2.resolve_mvars import (
6     InFluxesBySymbol,
7     OutFluxesBySymbol,
8     InternalFluxesBySymbol,
9     TimeSymbol,
10     StateVariableTuple,
11 )
12 import bgc_md2.resolve_computers as bgc_c
13
14 # Make a small dictionary for the variables we will use
15 sym_dict={}
16 r_vl_2_wl: "Internal flux rate from leaf to wood",
17 r_wl_2_vl: "Internal flux rate from wood to leaf",
18 C_soil_fast: "",
19 C_soil_slow: "",
20 C_soil_passive: "",
21 C_leaf: "",
22 C_root: "",
23 C_wood: "",
24 C_leaf_litter: "",
25 C_root_litter: "",
26 C_wood_litter: "",
27 r_C_leaf_2_C_wood_litter: "",
28 r_C_root_2_C_wood_litter: "",
29 r_C_wood_2_C_wood_litter: "",
30 r_C_leaf_litter_rh: "",
31 r_C_root_litter_rh: "",
32 r_C_wood_litter_rh: "",
33 r_C_soil_fast_rh: "",
34 r_C_soil_slow_rh: "",
35 r_C_soil_passive_rh: "",
36 r_C_leaf_litter_2_C_soil_fast: "",
37 r_C_leaf_litter_2_C_soil_slow: "",
38 r_C_leaf_litter_2_C_soil_passive: "",
39 r_C_wood_litter_2_C_soil_fast: "",
40 r_C_wood_litter_2_C_soil_slow: "",
41 r_C_wood_litter_2_C_soil_passive: "",
42 r_C_root_litter_2_C_soil_fast: "",
43 r_C_root_litter_2_C_soil_slow: "",
44 r_C_root_litter_2_C_soil_passive: "",
45 tau: "air temperature",
46 rho: "air",
47 T_B: "",
48 E: "",
49 RH: "",
50 beta_leaf: "",
51 beta_wood: "",
52
53 for k in sym_dict.keys():
54     code=k + Symbol('()').format(k)
55     exec(code)
56
57 # some we will also use some symbols for functions (which appear with an argument)
58 func_dict={}
59 x1: "a scalar function of temperature and moisture and thereby ultimately of time",
60 NPP: "",
61
62 for k in func_dict.keys():
63     code=k + Function('()').format(k)
64     exec(code)
65
66 t=TimeSymbol('t')
67 beta_root = 1.0. (beta_leaf+beta_wood)
68 mvs = OHVS(
69
70     StateVariableTuple((
71         C_wood,
72         C_leaf,
73         C_root,
74         C_leaf_litter,
75         C_wood_litter,
76         C_root_litter,
77         C_soil_fast,
78         C_soil_slow,
79         C_soil_passive,
80     )),
81     InFluxesBySymbol(
82         {
83             C_leaf: NPP(t) * beta_leaf,
84             C_root: NPP(t) * beta_root,
85             C_wood: NPP(t) * beta_wood
86         }
87     ),
88     OutFluxesBySymbol(
89         {
90             C_leaf_litter: r_C_leaf_litter_rh*C_leaf_litter*x1(t),
91             C_wood_litter: r_C_wood_litter_rh*C_wood_litter*x1(t),
92             C_root_litter: r_C_root_litter_rh*C_root_litter*x1(t),
93             C_soil_fast: r_C_soil_fast_rh*C_soil_fast*x1(t),
94             C_soil_slow: r_C_soil_slow_rh*C_soil_slow*x1(t),
95             C_soil_passive: r_C_soil_passive_rh*C_soil_passive*x1(t),
96         }
97     ),
98     InternalFluxesBySymbol(
99         {
100             (C_leaf, C_leaf_litter): r_C_leaf_2_C_wood_litter*C_leaf,
101             (C_wood, C_wood_litter): r_C_wood_2_C_wood_litter*C_wood,
102             (C_root, C_root_litter): r_C_root_2_C_root_litter*C_root,
103             (C_leaf_litter, C_soil_fast) : r_C_leaf_litter_2_C_soil_fast * C_leaf_litter*x1(t),
104             (C_leaf_litter, C_soil_slow) : r_C_leaf_litter_2_C_soil_slow * C_leaf_litter*x1(t),
105             (C_leaf_litter, C_soil_passive) : r_C_leaf_litter_2_C_soil_passive * C_leaf_litter*x1(t),
106             (C_wood_litter, C_soil_fast) : r_C_wood_litter_2_C_soil_fast * C_wood_litter*x1(t),
107             (C_wood_litter, C_soil_slow) : r_C_wood_litter_2_C_soil_slow * C_wood_litter*x1(t),
108             (C_wood_litter, C_soil_passive) : r_C_wood_litter_2_C_soil_passive * C_wood_litter*x1(t),
109             (C_root_litter, C_soil_fast) : r_C_root_litter_2_C_soil_fast * C_root_litter*x1(t),
110             (C_root_litter, C_soil_slow) : r_C_root_litter_2_C_soil_slow * C_root_litter*x1(t),
111             (C_root_litter, C_soil_passive) : r_C_root_litter_2_C_soil_passive * C_root_litter*x1(t),
112         }
113     )
114 ),
115     BibInfo(Bibliographical Information
116             name="VistIt",
117             longName=" ",
118             version="1",
119             entryAuthor="Konstantyn Viatkin",
120             entryAuthorOrCids=" ",
121             entryCreationDate=" ",
122             doi=" ")
123 )
```

Relation to other Python Packages



Applications

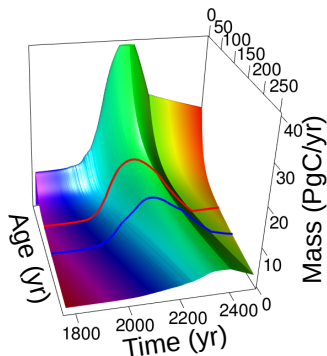


Figure: age distribution of a pool as function of time

Metzler, H., Müller, M., and Sierra, C. (2018). Transit-time and age distributions for nonlinear time-dependent compartmental systems. *Proceedings of the National Academy of Sciences*, 115:201705296.

Summary: Give me what you have and I'll show you what I can do with it

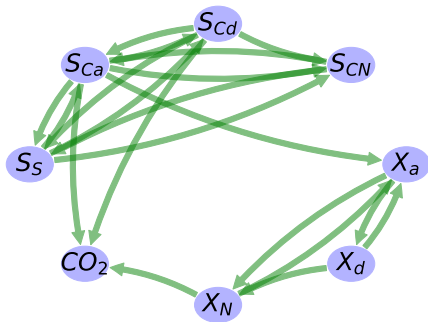
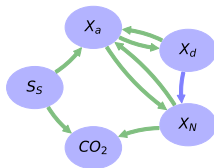
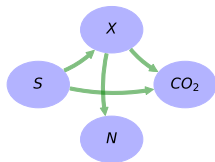
`bgc_md` is a library providing:

1. Datatypes defining building blocks of models e.g. `CompartmentalMatrix`, `InternalFluxesBySymbol`, ...
2. Functions operating on those properties (forming the edges of the graph where the Datatypes are nodes)
3. A user interface based on graph algorithms to
 - 3.1 compute the set of computable properties (e.g. the comparable criteria for a set of models, database queries)
 - 3.2 actually compute the desired properties by recursively connecting several function applications.
 - 3.3 show what is missing to compute a desired property.
4. 30+ vegetation, soil or ecosystem models for carbon and nitrogen cycling as reusable python modules using the building blocks in a flexible way.
5. An interface to *many algorithms* in `CompartmentalSystems` to compute diagnostic variables for *many models* in `bgc_md2`.

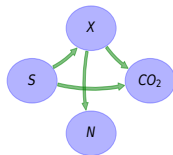
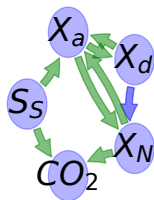
We might want to know ...

- ▶ If two or more (published) models are possibly possibly *identical*.
- ▶ If not, if they have some things in common and which.
- ▶ How they differ.
- ▶ If one is a generalization of another.
- ▶ If there are 'normal forms' e.g. views that hide incosequential details.
- ▶ Which model fits the data better.
- ▶ What constitutes a model. (Definitions may depend on the questions.)

Three Soil Models

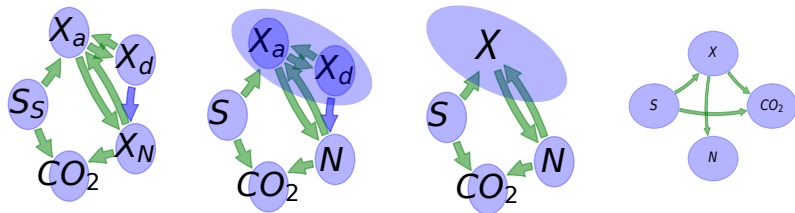


Symbolic comparison (pools)



Can be done by `bgc_md2` with a (python) dictionary for the variable names and a definition of the sets of pools that form the new superpools $\{X_a, X_d\} \rightarrow X$

Symbolic comparison (pools)



Can be done by `bgc_md2` with a (python) dictionary for the variable names and a definition of the sets of pools that form the new superpools $\{X_a, X_d\} \rightarrow X$

Symbolic comparison (equations)

- Translation of the ODE can also be assisted by `bgc_md2` with `sympy`.

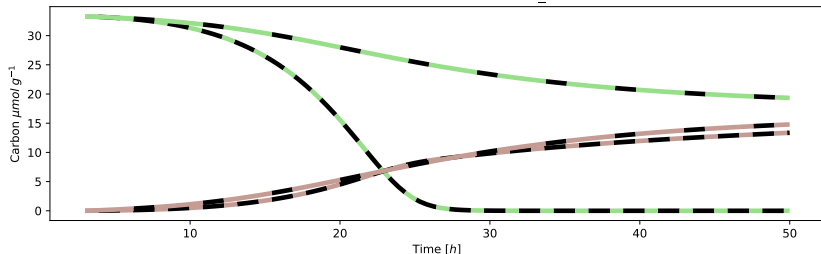
$$\left\{ \begin{array}{l} \dot{X}_a = f(X_a, X_d, X_n, S, \mathbf{p}) \\ \dot{X}_d = g(X_a, X_d, X_n, S, \mathbf{p}) \\ \vdots \\ X_a(t_0) = X_{a0} \\ X_d(t_0) = X_{d0} \\ \vdots \end{array} \right\} \xrightarrow[R = \frac{X_a}{X_a + X_d}]{} \left\{ \begin{array}{l} \dot{X} = h(X, R, N, S, \mathbf{p}) \\ \dot{R} = j(X, R, N, S, \mathbf{p}) \\ \vdots \\ X(t_0) = X_0 \\ R(t_0) = R_0 \\ \vdots \end{array} \right.$$

The constraints are an essential part of the model. (As we will see later)

Forward simulations with given parameters (easy)

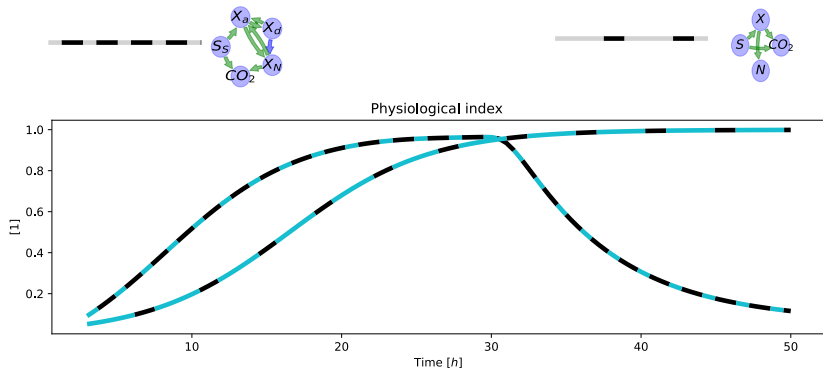


Substrate and accumulated CO_2



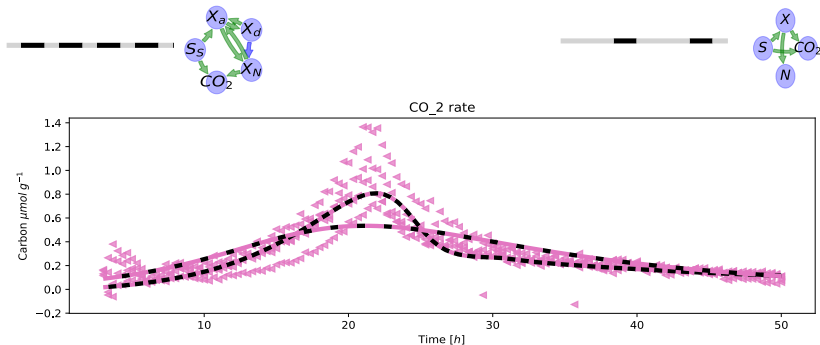
Automatic unit conversion (different models have different units)

Forward simulations with given parameters (easy) II



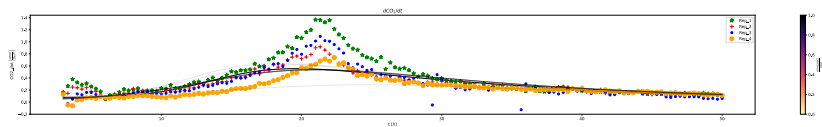
Lots of derived "diagnostic" variables, which can be computed for both models and allow comparison on a meta level.

Parameter estimation (hard)



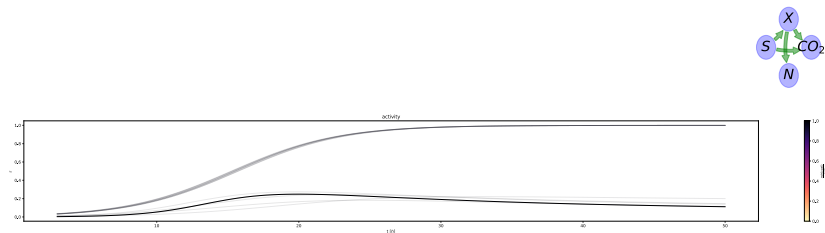
Hard because we have to be fair. Both models must have the same chance to hit the data.

Parameter estimation (hard) II



Two "best" fits.

Parameter estimation (hard) III



But very different behavior...

Possible reasons for unfairness

- ▶ Insufficient definition of model. E.g. **constraints**.
- ▶ Numerical instability. Two very different parameter combinations fit the data nearly equally well. (Identifiability problems)
- ▶ Dependence of qualitative results on hyperparameters.
 - ▶ Second best fit catches up and overtakes the best fit if we increase the number of iteration steps.
 - ▶

